Gain standard: PLC naming conventions

Document name:	Gain standard v2.4 - PLC naming conventions.docx
Status:	Final
Released by:	Gain Automation Technology B.V. hereafter named Gain
Author:	Dieter Kuiper
Last modified by:	Marco Conrads
Date:	12-08-2024

Table of content

1. Introduction

1.1. Revision history

Version	Date	Status	Comment	Initials
0.1	02-04-2021	Concept		MCo
0.2	18-05-2021	Concept	Changed layer structure to S88 naming	CdK
1.0	05-08-2021	Approved		MDu
1.1	05-11-2021	Revision	Restructured document outline	DKu
2.0	01-05-2022	Final	Revised naming convention, implemented S88 naming	DKu
2.1	23-05-2022	Final	Watermark added	WBo
2.2	02-12-2022	Final	Minor updates in chapter 1.2 & 2	WBo
2.3	13-01-2023	Final	Added rule chapter 3.1	WBo
2.4	12-08-2024	Final	Datatype any added	MCo

Table 1: Revision history

1.2. History and context

These are the coding guidelines for PLC projects. This document can be used by all engineers using the GBL.

It is a 'living' document. Remarks on this document will be appreciated to extend the knowledge which is placed in this document.

2. Gain PLC standard

This document is part of the “Gain PLC software standard”. This standard is separated in several documents, each document describes a part of the standard. These documents may be used individually and combined with other documents to create a customer specific standard.

A complete PLC software standard should (at least) contain the following topics:

- PLC naming convention
- PLC software structure
- PLC coding convention
- PLC version control

In this document the “PLC naming convention” is described.

3. PLC naming convention

PLC software consists of code, comments, programmable objects, and variable declarations. This section describes the naming conventions for the programmable objects (POU's) and the variable declarations.

3.1. Common naming rules

1. All software (code, comments, objects, variables, etc.) is written in English.
2. An object or variable name must be written in "PascalCasing".
3. An object or variable declaration must contain a type or instance identifier.
4. A variable declaration must contain a scope identifier, unless the scope is irrelevant or unknown (e.g., structured variables).
5. Structures or variables which are used in external communication must include a directional indication in the name.
6. Avoid using negatives and naming inverses.
7. Avoid ambiguous names. From the name it should be clear that a variable indicates an actual status or requests to a certain action.
8. Avoid the use of underscores in the object or variable name unless it improves clarity and readability.
9. If an object or variable name contains a hierarchy, let it start with the most global and end with the most specific name (hierarchical structures are preferred above hierarchical names).
10. A variable declaration should have a comment. Only when the variable name is self-explanatory, the comment can be omitted.
11. Only alphanumerical and underscore characters are allowed in naming.

3.2. Object declaration (POU's)

3.2.1. Type definition

All programmable objects should have a prefix representing the type of the object:

Prefix	*Object type*
OB_	Organization Block
DB_	Data block (not for instance data block)
PRG_	Program
FB_	Function Block
F_	Function
ST_	Structure or UDT
E_	Enumerated type
UN_	Function Block as unit
EM_	Function Block as equipment module
CM_	Function Block as control module
ITF_	Interface definition
GVL_	Global variable list

3.2.2. Object extension definition

If a programmable object has an extension, the following prefixes must be used:

Prefix	*Description*
a_	Action which can be called external or internal (public).
_a_	Action which can only be called internal (private).
m_	Method with public access specifier (external or internal use).
_m_	Method with private or protected access specifier (internal use).
p_	Property with public access specifier (external or internal use).
_p_	Property with private or protected access specifier (internal use).

3.3. Variable declaration

3.3.1. Scope definition

If a variable has a scope (e.g., unstructured), the following prefix must be used:

Prefix	*Description*
I_	Hardware input parameter.
Q_	Hardware output parameter.
i_	Input parameter for a function, function block or program.
q_	Output parameter for a function, function block or program.
iq_	In-Out parameter for a function, function block or program.
g_	Global variable.
s_	Local variable (static, with memory).
t_	Local variable for temporarily use (no memory).
c_	Constant variable (local or global).

3.3.2. Type definition

A variable can either be a common base type, an instance of an object or an instance of standard component.

3.3.2.1. Instances of elementary data types

Variables which represent a common base type, should have the following prefix:

Prefix	*Data type*
b	bool, bit
n	byte, int, uint, dint, udint, lint, ulint, word, dword, lword
f	real, lreal
s	string
tm	time
dt	date, datetime
p	Pointer

Examples:

s_bValue Local boolean variable.

c_nValue Constant integer variable.

3.3.2.2. Instances of objects (POU's)

Variables which are an instance of a POU, should have the following prefix:

Prefix	*Description*
fb	Instance for function block 'FB_xxx'.
un	Instance for unit 'UN_xxx'.
em	Instance for equipment module 'EM_xx'.
cm	Instance for control module 'CM_xxxx'.
st	Instance for a structure 'ST_xxx'.
e	Instance for an enumerator 'E_xxx'.
itf	Reference to an instance of a function block with interface 'ITF_xxx'.

Examples:

s_cmMotorM101 Control module with instance/variable name 'MotorM101'.

s_eUnitStatus Enumerator with instance/variable name 'UnitStatus'.

3.3.2.3. Instances of standard components

Variables which are an instance of a standard component, should be prefixed as:

Prefix	*Component*
rs	reset-set
sr	set-reset
ton	on-delay timer
tof	off-delay timer
osr	one-shot rising (rising edge trigger)
osf	one-shot falling (falling edge trigger)
cu	counter up
cd	counter down
cud	counter up-down

Examples:

s_tonReady Local timer on-delay named 'Ready'.

s_osrResetButton Local rising edge trigger named 'ResetButton'.

3.3.2.4. Extended type definitions

Variables which are type extensions, should have the following prefix:

Prefix	*Data type*
a	array of …
p	pointer to …
r	reference to …

x	Any

Examples:

s_anValues	Local array of integer data types.
i_pstStatus	Input pointer to a structured data type.

3.3.3. External communication

Structures which are used for external communication (other machines or an HMI) should include directional notation. If the structure is bi-directional (read & write) the variables inside the structure should indicate the direction. Bi-directional variables should be avoided as much as possible.

Examples:

g_stPmlTagsToXXX	Global PML Tags which are send to machine 'XXX'.
g_stPmlTagsFromXXX	Global PML Tags which are received from machine 'XXX'.
s_stBtnStop.nToHmi	Button status towards HMI
s_stBtnStop.nFromHmi	Button pressed event from HMI

3.3.4. Standard abbreviations

If applicable, variables can be abbreviated using the following prefixes:

Prefix	***Description***
Act	Actual
Alm	Alarm
Alw	Allow
Cfg	Configuration
Cmd	Command
Err	Error
Ilck	Interlock
Req	Request
Set	Setting
Sta	Status
Wrn	Warning

Examples:

i_bReqStart	Request to start action.
q_bAlwStart	Allowed to start action.
i_nCfgMaxRetries	Configuration input for 'MaxRetries'.
s_bStaRunning	Status running.

Appendix A: Naming examples

In this appendix several examples of the Gain naming conventions are provided. These examples are meant for explanatory purposes. It should be named that ideal object/variable names are always subjective and open to discussion.

A.1 Object declarations

In this section examples of PLC objects are shown.

Object name	*Type*	*Example usage*
PRG_Main	Program	Main PLC program.
GVL_Constants	Global variable list	Global area containing common constants
DB_Constants	Data block	Data block containing common constants
F_fLimit	Function	Function to limit floating point variables
FB_SeqTimer	Function block	Function block to measure step times.
EM_Conveyor	Equipment module	
E_EM_Conveyor_SeqExecute	Enumerator	Execute sequence steps for the equipment module conveyor.
ST_EM_Conveyor_HMI	Struct	Structure containing the HMI variables for the equipment module conveyor.
CM_Motor	Control module	
ST_CM_Motor_Config	Struct	Structure containing the configuration settings for control module CM_Motor.
CM_Motor.m_Start	Public method	Method to start the motor
CM_Motor.p_fPosition	Public property	Property of motor for the position value.

A.2 Variable declarations

In this section examples of PLC variables are shown.

Variable name	*Scope*	*Type*	*Example usage*
I_bBtnStart	Hardware input	bool	Start button
I_nValue	Hardware input	dword	Raw value from an analog input card
Q_bLedStartBtn	Hardware output	bool	Led
Q_stControlWord	Hardware output	struct	Struct to control an external drive
i_bCmdReset	local input	bool	Command to reset
i_pstRecipe	local input	pointer to struct	Input pointer to recipe struct
q_bStaEnabled	local output	bool	Enabled status
s_fActPosition	local variable	real/lreal	Actual position
s_tonStandstill	local variable	on-delay timer	On delay timer for standstill
t_nIndex	temp variable	int	Variable for looping in an array.
g_acmConveyors	global variables	array of CM	Global array of control modules